

Designing REST API

Lecture Notes

Topic: Designing REST API Using Case Study

Duration: 2 Hours

Lecture Type: Theory + Practical (Step-by-Step)

Level: Computer Science Students

Introduction to REST API (0 – 20 Minutes)

What is an API?

API (Application Programming Interface) is a set of rules that allows different software systems to communicate with each other.

Real-life Example:

- When you use **Foodpanda** or **Careem**, the mobile app communicates with a server using APIs.
 - The app sends a request:
 "Show available restaurants"
 - The server responds with restaurant data in **JSON format**.
-

What is REST?

REST (Representational State Transfer) is an architectural style for designing networked applications.

It uses:

- HTTP protocol
- Standard HTTP methods
- Stateless communication

REST Principles:

1. Stateless
 2. Client-Server Architecture
 3. Uniform Interface
 4. Resource-based
-

HTTP Methods Used in REST

Method	Purpose	Real-life Example
GET	Retrieve data	View product
POST	Create data	Register user
PUT	Update data	Update profile
DELETE	Delete data	Delete account

Student Activity 1 (After 20 Minutes)

Think-Pair-Share Activity

Ask students:

- What happens when you login to Facebook?
- Which HTTP method is used?
- Is login GET or POST? Why?

 Expected Answer: POST (because credentials are sent securely in request body)

Case Study Introduction (20 – 40 Minutes)

Case Study: Online Bookstore REST API

We will design a REST API for an **Online Bookstore System**.

System Requirements:

The system should allow:

1. User registration & login
2. Add new books
3. View all books
4. Update book details
5. Delete book

6. Place orders

Step 1: Identify Resources

In REST, everything is a **Resource**.

For Online Bookstore:

- Users
- Books
- Orders

Step 2: Define Endpoints

Users

Operation	Method	Endpoint
Register User	POST	/users
Get All Users	GET	/users
Get Single User	GET	/users/{id}
Update User	PUT	/users/{id}
Delete User	DELETE	/users/{id}

Books

Operation	Method	Endpoint
Add Book	POST	/books
Get All Books	GET	/books
Get Book	GET	/books/{id}
Update Book	PUT	/books/{id}
Delete Book	DELETE	/books/{id}

Orders

Operation	Method	Endpoint
Create Order	POST	/orders
Get Orders	GET	/orders

Student Activity 2 (40 Minutes)

Quick Quiz (5 minutes)

1. What is the endpoint to update book with id = 10?
2. Which HTTP method is used to delete a user?
3. Is `/getAllBooks` a good REST endpoint? Why?

👉 Discussion:

- REST uses nouns, not verbs.
- Correct: `/books`

REST API Design Best Practices (40 – 60 Minutes)

1. Use Nouns, Not Verbs

✗ `/getBooks`

✓ `/books`

2. Use Plural Resource Names

✗ `/book`

✓ `/books`

3. Use Proper HTTP Status Codes

Code	Meaning
200	OK
201	Created
400	Bad Request
404	Not Found
500	Server Error

4. Use JSON Format

Example Response:

```
{
  "id": 1,
  "title": "Clean Code",
  "author": "Robert C. Martin",
  "price": 500
}
```

Student Activity 3 (After 60 Minutes – 10 min break discussion)

Group Activity:

Design REST endpoints for:

 University Management System

Resources:

- Students
- Courses
- Teachers

Each group presents their endpoints.

Practical Implementation (Step-by-Step) (60 – 100 Minutes)

We will implement using:

- Node.js
 - Express.js
 - Postman
-

Step 1: Initialize Project

```
npm init -y
```

Install dependencies:

```
npm install express body-parser
```

Step 2: Create Server

```
const express = require('express');
const app = express();

app.use(express.json());

app.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

Step 3: Create Books API

GET All Books

```
let books = [];  
  
app.get('/books', (req, res) => {  
  res.json(books);  
});
```

POST Book

```
app.post('/books', (req, res) => {  
  const book = req.body;  
  books.push(book);  
  res.status(201).json(book);  
});
```

GET Book by ID

```
app.get('/books/:id', (req, res) => {  
  const id = parseInt(req.params.id);  
  const book = books.find(b => b.id === id);  
  
  if (!book) {  
    return res.status(404).json({message: "Book not found"});  
  }  
  
  res.json(book);  
});
```

PUT Update Book

```
app.put('/books/:id', (req, res) => {  
  const id = parseInt(req.params.id);  
  const index = books.findIndex(b => b.id === id);  
  
  if (index === -1) {  
    return res.status(404).json({message: "Book not found"});  
  }  
  
  books[index] = req.body;  
  res.json(books[index]);  
});
```

DELETE Book

```
app.delete('/books/:id', (req, res) => {  
  const id = parseInt(req.params.id);  
  books = books.filter(b => b.id !== id);  
  res.json({message: "Book deleted"});  
});
```

Student Activity 4 (Hands-on Practice – 15 Minutes)

Students must:

1. Add 3 books using POST
2. Retrieve all books
3. Update one book

4. Delete one book

Use Postman to test.

Advanced Concepts (100 – 115 Minutes)

1. Query Parameters

Example:

`GET /books?author=Ali`

2. Pagination

`GET /books?page=1&limit=10`

3. Authentication (Basic Concept)

Use JWT tokens for secure APIs.

Login Flow:

1. User sends credentials
 2. Server verifies
 3. Server sends token
 4. Client sends token in header
-

Final Class Discussion (115 – 120 Minutes)

Ask students:

1. Why REST APIs are stateless?
 2. What happens if we don't use proper HTTP status codes?
 3. What improvements can we add to our bookstore API?
-

Summary of Today's Lecture

Students learned:

- ✓ What REST API is
- ✓ REST principles
- ✓ Designing endpoints
- ✓ HTTP methods
- ✓ Status codes

- ✓ Practical implementation in Node.js
 - ✓ Testing with Postman
-

Assignment

Design REST API for:

 E-Commerce System

Resources:

- Customers
- Products
- Cart
- Orders
- Payments

Draw:

- Endpoints
 - HTTP Methods
 - Sample JSON
-

References (For Academic Integrity)

- REST architectural style defined by Roy Fielding (2000 doctoral dissertation)
- HTTP Protocol (RFC 2616)

If you need, I can also provide:

- PowerPoint slides
- Lab manual version
- ER diagram for bookstore
- Database integration version (MongoDB/MySQL)
- JWT authentication full implementation